

Production-Grade RAG Portfolio Projects — 2026/2027 Hiring Cycle

Five project ideas, fully specced, ranked, and one final recommendation. Built around the advanced RAG techniques you already have in hand: hybrid search, RRF, multi-query retrieval, reranking, semantic/agentic chunking, multi-modal RAG, and history-aware retrieval. Each project is designed so none of that knowledge goes to waste — and so you add the MLOps/production layer that separates a tutorial project from a hire-me project.

Project 1: Financial Research Intelligence Platform

Problem Statement

Equity research analysts and finance teams spend hours manually reading 10-Ks, 10-Qs, earnings call transcripts, and investor decks to answer questions like "How did Nvidia's data center revenue trend over the last 4 quarters, and what did management say about supply constraints?" Answers live across dense PDFs with tables, charts, and unstructured commentary — no single source has both the number and the narrative.

Why Companies Care

Bloomberg, FactSet, AlphaSense, and every major bank's internal tools team are racing to build exactly this. Fintech AI hiring (analyst copilots, document intelligence) is one of the hottest GenAI niches in 2026 because the ROI is provable in analyst-hours saved, and the data is genuinely hard (tables + numbers + qualitative text + multi-year comparisons).

Real Users

Equity research analysts, hedge fund analysts, retail investors (B2C version), corporate finance/FP&A teams.

RAG Architecture

- **Ingestion:** PDF parsing with layout-aware extraction (tables vs. prose vs. footnotes treated differently)
- **Multi-modal RAG:** tables converted to structured representations (not flattened text) + chart/image captioning for embedded charts
- **Chunking:** agentic chunking for narrative sections (MD&A, risk factors), table-aware chunking for financial statements (never split a table mid-row)
- **Retrieval:** hybrid search (BM25 for ticker/exact-number lookups + dense embeddings for semantic "how did X trend") combined via RRF
- **Reranking:** Cohere rerank to resolve cases where the same metric appears in multiple quarters' filings
- **History-aware retrieval:** for multi-turn "and how does that compare to last year?" follow-ups

- **Generation:** numeric answers are extracted and verified against the source table (not hallucinated by the LLM), with citations pointing to exact filing + page

Advanced Features

- Quarter-over-quarter / YoY automatic comparison agent
- "Show your work" mode — displays the exact table cell or transcript sentence used
- Multi-document synthesis across 10-Ks + earnings calls + analyst notes
- Confidence scoring on numeric answers (flag when retrieval confidence is low)

Dataset Sources

- SEC EDGAR (free, full 10-K/10-Q filings)
- Earnings call transcripts (Motley Fool, Seeking Alpha excerpts, or scrape investor relations pages — check ToS)
- Synthetic/sample analyst notes if real ones are inaccessible

Tech Stack

FastAPI backend, LangChain/LlamaIndex orchestration, ChromaDB or Qdrant (better metadata filtering for tickers/dates/quarters), Cohere rerank, HuggingFace or OpenAI embeddings, `unstructured` or `pymupdf` + `camelot`/`pdfplumber` for table extraction, React or Streamlit frontend, Docker, deployed on Render/Fly.io/AWS.

Deployment Strategy

Containerize ingestion pipeline separately from the query API (ingestion is batch, querying is real-time). Use a managed vector DB (Qdrant Cloud free tier) to avoid self-hosting headaches early on. Cache frequent queries (e.g., "latest revenue") with TTL since filings update quarterly, not real-time.

Challenges and Edge Cases

- Tables spanning multiple PDF pages
- Restated financials (same metric, different filing, different number — must surface both with dates)
- Numbers in different units (millions vs. thousands) across documents
- Hallucination risk on numeric answers — needs strict source-grounding, not generative math

Resume Impact

Very high — demonstrates multi-modal RAG (rare on resumes), domain-specific accuracy engineering, and a problem hiring managers immediately recognize as commercially valuable.

Difficulty Level: 8/10

Estimated Development Time: 5-7 weeks (part-time alongside coursework)

How to Scale It for Production

Add incremental ingestion (new filings auto-trigger re-embedding), async task queue (Celery/Redis) for PDF processing, horizontal scaling of the API layer, and a feedback loop where flagged wrong answers get added to an eval set.

How to Convert It into an MLOps + GenAI Portfolio Project

Add RAGAS or DeepEval for automated retrieval/answer-quality evaluation, LangSmith or Phoenix (Arize) for tracing every retrieval + generation step, a small eval dataset of analyst Q&A pairs with gold answers, CI pipeline that runs the eval suite on every chunking/retrieval change, and a Grafana dashboard tracking latency, retrieval hit-rate, and answer groundedness over time.

Project 2: Enterprise Engineering Knowledge Copilot (Glean-style)

Problem Statement

At any company past ~50 engineers, knowledge fragments across Confluence, Slack, GitHub PRs/issues, and internal wikis. New engineers and even senior ones waste hours asking "has someone already solved this?" or "why was this architectural decision made?"

Why Companies Care

This is literally Glean's (\$2B+ valuation), Notion AI's, and Microsoft Copilot's core bet — internal enterprise search/synthesis is one of the largest GenAI markets right now. Every mid-to-large company is either buying or building this. ServiceNow, Salesforce, Atlassian, and Microsoft all have GenAI teams hiring specifically for this.

Real Users

Software engineers, engineering managers, new hires during onboarding, support engineers escalating tickets.

RAG Architecture

- **Multi-source ingestion:** GitHub (code + PR descriptions + issues), Slack export/API, Markdown wikis, Notion exports
- **Source-aware chunking:** code uses AST-aware chunking (function/class boundaries), prose uses semantic chunking, Slack threads chunked by conversation, not message
- **Hybrid search:** BM25 crucial here (engineers search exact function/error-message names — pure semantic search fails on this)
- **Multi-query retrieval:** expands ambiguous queries ("why is the build slow") into sub-queries across different source types

- **Reranking:** cross-source reranking so code results don't drown out the relevant Slack discussion explaining *why*
- **History-aware retrieval:** critical for the natural "ok but what about staging?" follow-up pattern

Advanced Features

- Source attribution UI (this answer combines a Slack thread from March + a PR comment + a wiki page)
- Freshness weighting (recent Slack discussion should often outrank a 2-year-old outdated wiki page)
- Access-control-aware retrieval (simulate permission boundaries — a strong signal for enterprise-readiness)
- "Stale doc" detector — flags wiki pages contradicted by recent code changes

Dataset Sources

Use a real open-source project's GitHub repo + issues + PRs (e.g., a mid-size repo like FastAPI or LangChain itself) as a stand-in for "internal" data, plus synthetic Slack-style discussion threads you generate to simulate the conversational layer.

Tech Stack

FastAPI, LangChain, Qdrant or Weaviate (good hybrid search support natively), GitHub API, Cohere rerank, React frontend with source-type filtering, Docker Compose for multi-service local dev, deployed on AWS/GCP with a managed vector DB.

Deployment Strategy

Multi-container setup (ingestion workers per source type, query API, vector DB), background sync jobs (cron or event-driven via GitHub webhooks) to keep the index fresh — this freshness story is a key talking point in interviews.

Challenges and Edge Cases

- Conflicting information across sources (old wiki says X, recent PR does Y)
- Code chunks losing context without surrounding file/module info
- Slack threads with no clear resolution
- Rate limits on GitHub/Slack APIs during ingestion

Resume Impact

Very high — this maps almost 1:1 to active job descriptions at Glean, Atlassian, ServiceNow, Microsoft, and most enterprise SaaS GenAI teams. Interviewers will recognize the problem instantly.

Difficulty Level: 7/10

Estimated Development Time: 4-6 weeks

How to Scale It for Production

Event-driven ingestion (webhooks instead of polling), per-tenant vector namespaces for multi-org support, query result caching, async reranking pipeline to keep p99 latency low under load.

How to Convert It into an MLOps + GenAI Portfolio Project

Build an eval set of real engineering questions with known-good answers, track retrieval precision/recall per source type, add LangSmith tracing, set up a nightly eval job in GitHub Actions that regression-tests retrieval quality, and add a simple A/B harness comparing chunking strategies (semantic vs. agentic vs. recursive) with quantified results — this comparison alone is a great blog post / LinkedIn article.

Project 3: Legal Contract Intelligence & Compliance Assistant

Problem Statement

Legal and compliance teams manually review hundreds of contracts (NDAs, vendor agreements, MSAs) to check for non-standard clauses, missing terms, or regulatory risk. This is slow, expensive (outside counsel billable hours), and error-prone at scale.

Why Companies Care

Legal tech (Ironclad, Harvey AI, Spellbook) is one of the fastest-growing GenAI verticals, with massive funding and active hiring. Every company with a legal team is a potential buyer — this has obvious enterprise value and a clear ROI story (hours of lawyer time saved).

Real Users

In-house legal teams, contract managers, compliance officers, paralegals.

RAG Architecture

- **Ingestion:** contract PDFs/DOCX, clause-level structure preserved
- **Agentic chunking:** contracts are chunked by clause/section using an LLM pass to identify logical boundaries (critical — naive splitting breaks legal meaning)
- **Retrieval:** hybrid search for exact clause-type lookups ("indemnification clause") + semantic search for "find contracts with unusual termination terms"
- **Multi-query + RRF:** a single compliance question often needs comparing the same clause type across many contracts
- **Reranking:** surfaces the most relevant clause among dozens of similar-sounding ones

- **Standard-clause comparison:** a "golden" clause library compared against each contract's actual clause via embedding similarity, flagging deviations

Advanced Features

- Automated risk-flagging (deviations from standard clause library beyond a similarity threshold)
- Clause-type classifier as a pre-retrieval filter
- Redline-style diff view between standard clause and the found clause
- Batch mode: upload 50 contracts, get a compliance summary table

Dataset Sources

CUAD (Contract Understanding Atticus Dataset) — a real, public, labeled legal contract dataset built exactly for this use case. This is a major advantage: real data, real labels, no scraping headaches.

Tech Stack

FastAPI, LangChain, ChromaDB/Pinecone, Cohere rerank, spaCy or a legal-NER model for entity extraction, Streamlit (fast to build) or React (more impressive) frontend, Docker, deployed on Render or AWS.

Deployment Strategy

Document-level isolation (each uploaded contract set is its own logical namespace for privacy realism), async processing for batch contract review jobs, exportable PDF/Excel compliance reports as a deliverable users can actually use.

Challenges and Edge Cases

- Legal language is dense and context-dependent — clause meaning can flip on a single negation
- Cross-references within a contract ("as defined in Section 4.2")
- OCR errors in scanned older contracts
- Jurisdiction-specific clause variations

Resume Impact

High, especially if targeting AI startups in legal tech specifically. CUAD as a dataset signals rigor since it's a recognized academic/industry benchmark.

Difficulty Level: 7/10

Estimated Development Time: 4-5 weeks

How to Scale It for Production

Multi-tenant document isolation, parallelized batch processing queue, versioned clause-library updates, audit logging (compliance products need defensible audit trails — a great

thing to mention in interviews).

How to Convert It into an MLOps + GenAI Portfolio Project

Use CUAD's labels to build a quantitative retrieval-accuracy benchmark (precision@k on clause retrieval), track this in an eval dashboard, add model/version tracking via MLflow if you experiment with different embedding models, and write up a "before/after" reranking improvement with real numbers — recruiters love a number.

Project 4: Multi-Tenant Customer Support Agentic RAG (SaaS Copilot)

Problem Statement

SaaS companies' support teams answer the same questions repeatedly across product docs, past tickets, and changelogs, while also needing to know when a question requires escalation rather than an automated answer.

Why Companies Care

This is the exact product category of Intercom Fin, Zendesk AI, and dozens of support-AI startups. It's heavily funded and hiring aggressively because support cost reduction is an easy budget line to point to.

Real Users

Support agents (as a copilot, not full replacement), end customers (self-serve), support team leads (analytics).

RAG Architecture

- **Multi-tenant design:** each company's docs/tickets are isolated by namespace — a core enterprise SaaS requirement to demonstrate
- **Hybrid search + RRF:** across product docs, FAQ, and resolved ticket history
- **History-aware retrieval:** essential for multi-turn support conversations
- **Agentic routing layer:** an agent decides whether to answer directly, ask a clarifying question, or escalate to a human — this is what makes it "agentic RAG" rather than a chatbot
- **Reranking:** tuned specifically to prioritize recently-resolved tickets over outdated docs

Advanced Features

- Escalation-confidence scoring (model self-assesses whether it should hand off to a human)
- Ticket-resolution feedback loop (closed tickets become new retrieval sources)
- Per-tenant analytics dashboard (deflection rate, top unanswered questions — directly maps to business KPIs)

- Sentiment-aware response tone adjustment

Dataset Sources

Public support datasets (e.g., Twitter customer support datasets, Zendesk-style sample data) or generate a realistic synthetic SaaS product doc + ticket corpus for a fictional product — gives you full control over edge cases to demo.

Tech Stack

FastAPI, LangChain/LangGraph (for the agentic routing logic), Qdrant, Cohere rerank, React frontend with a dual view (customer chat + agent analytics dashboard), Docker, deployed on AWS/GCP with multi-tenant data isolation.

Deployment Strategy

Namespace-per-tenant in the vector DB, API-key-based tenant routing, rate limiting per tenant, a separate analytics service consuming logged interactions.

Challenges and Edge Cases

- Knowing when *not* to answer (escalation logic is harder than retrieval)
- Tenant data leakage risk (a strong "I built this securely" talking point)
- Stale product docs after a feature update
- Ambiguous queries needing clarification before retrieval

Resume Impact

High — the multi-tenant + agentic routing angle directly demonstrates production SaaS thinking, not just RAG mechanics. Strong fit for GenAI Engineer roles at support-tech and general SaaS companies.

Difficulty Level: 7/10

Estimated Development Time: 4-6 weeks

How to Scale It for Production

Tenant onboarding automation (auto-create namespace + ingest docs on signup), async ticket ingestion pipeline, per-tenant rate limiting and cost tracking, canary-testing new prompt/retrieval configs on a small tenant subset before full rollout.

How to Convert It into an MLOps + GenAI Portfolio Project

Track deflection rate and escalation accuracy as core ML metrics, build a feedback-labeling loop (agents mark AI answers as helpful/unhelpful, feeding a fine-tuning or few-shot improvement cycle), add LangSmith/Phoenix tracing on the agentic routing decisions specifically (this is the most "ML engineering" part — evaluating an agent's decision quality, not just retrieval quality).

Project 5: Codebase + Technical Documentation RAG for Developer Tools

Problem Statement

Developers using a large SDK/API (think Stripe, AWS, or an internal platform team's tooling) constantly context-switch out of their IDE to search scattered docs, GitHub issues, and Stack Overflow-style answers.

Why Companies Care

Every devtools company (GitHub/Copilot, Sourcegraph, AWS, Stripe, internal platform teams everywhere) invests heavily here because reducing developer friction directly drives product adoption. This is squarely in Nvidia/AWS/Microsoft devtools hiring territory.

Real Users

Developers integrating an API/SDK, internal platform engineering teams, DevRel teams.

RAG Architecture

- **Code-aware chunking:** AST-based chunking for source code, recursive chunking for docs/prose
- **Hybrid search:** BM25 essential for exact API/function name matches, dense embeddings for "how do I do X" conceptual queries
- **Multi-query retrieval:** expands vague developer questions into specific API-surface sub-queries
- **Reranking:** prioritizes official docs over GitHub issue discussions unless the issue directly resolves a known bug
- **Code-execution-aware generation:** generated code snippets validated against the actual SDK signature (prevents hallucinated parameters — a known failure mode worth solving explicitly)

Advanced Features

- IDE-style inline answer widget (VS Code extension as a stretch goal — huge resume differentiator)
- Auto-detection of SDK version drift (flags when docs reference a deprecated method)
- "Show me a working example" mode pulling from real repo code, not generated from scratch

Dataset Sources

Pick a real open-source SDK with public docs + GitHub repo (e.g., `requests`, `pandas`, `LangChain` itself) — gives you authentic, large-scale, well-structured data with no scraping risk.

Tech Stack

FastAPI, LangChain, ChromaDB, tree-sitter for AST-based code chunking, Cohere rerank, optionally a VS Code extension (TypeScript) calling your API, Docker, deployed on Fly.io/Render.

Deployment Strategy

CLI tool + API + optional IDE extension as three access points to the same backend, doc re-ingestion triggered on new SDK releases via GitHub webhook.

Challenges and Edge Cases

- Code chunks losing necessary import/context lines
- Multiple SDK versions with breaking changes
- Distinguishing "official" answers from community workarounds
- Generated code needs to be syntactically valid, not just semantically plausible

Resume Impact

High, particularly for devtools/platform engineering-adjacent GenAI roles (AWS, Nvidia, Sourcegraph, GitHub). The VS Code extension stretch goal alone is a strong differentiator most candidates won't have.

Difficulty Level: 6/10 (8/10 with the IDE extension)

Estimated Development Time: 3-5 weeks (add 2 weeks for the extension)

How to Scale It for Production

Version-pinned doc indices (serve the right docs for the SDK version a user has installed), incremental re-indexing on new releases, caching common API-lookup queries.

How to Convert It into an MLOps + GenAI Portfolio Project

Build an eval set of real developer questions with known correct API calls, measure code-correctness rate (does the generated snippet actually run against the real SDK?) as your core metric — this is a much stronger, more concrete eval story than generic "answer quality," add CI that runs generated code snippets in a sandbox to verify they execute without errors.

Ranking Across All Five Projects

Project	Resume Value	Hiring Market Demand	Learning Value	Technical Depth	Portfolio Uniqueness
1. Financial Research Intelligence	9	8	9	9	9
2. Enterprise Knowledge Copilot	9	9	8	8	7
3. Legal Contract Intelligence	8	7	7	7	8
4. Support Agentic RAG (SaaS)	8	9	8	7	6
5. Codebase/Devtools RAG	7	7	7	7	8 (9 with IDE extension)

Overall ranking (weighted toward resume value + technical depth):

1. Financial Research Intelligence Platform
 2. Enterprise Knowledge Copilot
 3. Support Agentic RAG (SaaS)
 4. Legal Contract Intelligence
 5. Codebase/Devtools RAG

Final Recommendation: Financial Research Intelligence Platform

For a 3rd-year student targeting GenAI Engineer, RAG Engineer, or MLOps Engineer roles in 2026–2027, **Project 1 is the strongest single pick**, for a few concrete reasons:

- **It forces you to use every advanced technique you already know in a way that's non-optional, not decorative.** Multi-modal RAG (tables + charts), hybrid search, RRF, agentic chunking, reranking, and history-aware retrieval all solve genuine problems here — there's no "I added reranking because the tutorial did" energy. Interviewers can tell the difference.
- **The eval story is unusually strong.** Financial numbers are objectively verifiable — you can build a real precision/accuracy benchmark ("94% of extracted Q3 revenue figures matched the source table exactly") rather than vague "the answers seem good" claims. That kind of measurable rigor is exactly what separates a CV bullet that gets a callback from one that doesn't.

- **It directly demonstrates the hardest unsolved RAG problem in industry right now: grounding numeric/tabular answers and not hallucinating them.** This is a live pain point at Bloomberg, AlphaSense, and every bank's internal AI team — naming it explicitly in an interview signals you understand production RAG failure modes, not just RAG happy paths.
- **It's extensible into a genuinely portfolio-worthy MLOps story:** incremental ingestion as new filings drop, eval regression testing in CI, and an observability dashboard — a believable "here's how I'd run this in production" narrative, which is what MLOps interviewers actually probe for.

A realistic build order: get a single-document Q&A pipeline working on a few 10-Ks first (don't multi-modal everything at once), then add table-aware chunking, then hybrid search + rerank, then the multi-quarter comparison agent, then the eval/observability layer last. That sequencing keeps you shipping something demo-able every week rather than stuck mid-build for a month.

If you want a slightly lower-effort but still very strong second choice, the **Enterprise Knowledge Copilot** has higher raw market demand (Glean-style products are everywhere right now) and is a touch faster to build since the data wrangling is less finicky than financial PDFs.